

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ
Военный факультет
Кафедра электронных вычислительных машин
Дисциплина: Жизненный цикл разработки программного обеспечения

Лабораторная работа №1
по дисциплине «Жизненный цикл разработки программного обеспечения»

Студент:
Преподаватель:

А.К. Масюк
С.С. Силич

МИНСК 2026

1 ЦЕЛЬ РАБОТЫ:

- изучить методы работы с одной из выбранных студентом сред разработки: IntelliJ IDEA (рекомендуемая среда), Eclipse, Microsoft Visual Studio Code, учитывая, что использовать можно только объектно-ориентированный язык программирования (C++, C#, Java, другой язык по согласованию с преподавателем);
- научиться разрабатывать в выбранной среде приложение с графическим интерфейсом;
- изучить MVC с активной и пассивной моделью.

2 ЗАДАНИЕ К ЛАБОРАТОРНОЙ РАБОТЕ

Разработать калькулятор веса на разных планетах. Человек вводит свой вес на Земле, программа должна рассчитать, сколько он будет весить на других планетах солнечной системы (Меркурий, Нептун, и т.д). На главной форме пусть будет кнопка «Ввести данные». По нажатию на нее пусть откроется дополнительное окно для ввода информации. Последние введенные данные нужно запоминать в главной форме и восстанавливать при повторном вводе. После этого на главной форме отобразить результат вычислений. Если введены некорректные данные – показать сообщение об ошибке. Добавить каркас MVC с активной моделью. Неудачникам студентам применить MVC с пассивной моделью. Для совсем неудачников студентов – из паттерна MVC реализовать класс контроллер, совмещенный с пассивной моделью.

3 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Структура и компоненты MVC

Model-View-Controller (MVC) – это фундаментальный шаблон проектирования, предназначенный для разделения бизнес-логики приложения (компонентов работы с данными) от его визуального представления.

Согласно классической концепции, архитектура приложения разделяется на три изолированных класса:

- Модель (Model) – управляет внутренним состоянием, данными и бизнес-логикой домена приложения. Она отвечает на запросы информации о своем состоянии и обрабатывает инструкции по его изменению. Модель является полностью независимым компонентом: она ничего не знает ни о Представлении, ни о Контроллере.
- Представление (View) – отвечает за графический интерфейс (GUI), визуальное отображение данных пользователю и презентационную логику.
- Контроллер (Controller) – перехватывает действия пользователя (нажатия клавиш, клики мыши), интерпретирует их и вызывает соответствующие методы Модели для изменения её состояния.

Активная и пассивная модели в MVC

В зависимости от способа синхронизации данных и интерфейса автор паттерна Стив Бурбек (Steve Burbeck) выделяет две ключевые модификации MVC:

- Пассивная модель (Passive Model):

Используется в архитектурах, где Контроллер является единственным источником изменений Модели. В этом сценарии Модель полностью автономна и не способна сообщать о смене своего состояния. Контроллер сначала модифицирует Модель, а затем самостоятельно и принудительно дает команду Представлению обновить отображение на экране. Классическим примером такой схемы является стандартный протокол HTTP в веб-приложениях.

- Активная модель (Active Model):

Применяется, когда состояние Модели может меняться без прямого участия текущего Контроллера (например, при поступлении внешних данных, вычислениях по таймеру или многопоточной обработке). Так как только сама Модель способна зафиксировать изменения своего внутреннего состояния, она обязана уведомить интерфейс о необходимости синхронизации. Чтобы сохранить независимость Модели от конкретных окон интерфейса, активная модель реализуется через паттерн Observer («Наблюдатель»), также известный как схема «издатель-подписчик» (*publish-subscribe*). Представления регистрируются в Модели в качестве наблюдателей. При изменении бизнес-данных Модель циклически обходит список подписчиков и автоматически генерирует для них сигналы оповещения, изолируя логику расчета от визуального кода.

Современная IDE (на примере связки VS Code и инструментария GCC/MinGW) — это комплекс программных средств для автоматизации разработки, состоящий из следующих компонентов:

- Текстовый редактор кода — оболочка с функциями подсветки синтаксиса и структурирования исходных файлов.
- Компилятор и Линкер (g++.exe) — транслятор кода C++ в машинные объектные модули и компоновщик их с системными библиотеками в файл .exe.
- Система автоматизации сборки (tasks.json) — диспетчер, управляющий последовательностью компиляции и передачи флагов оптимизации.
- Отладчик (gdb.exe) — инструмент для пошагового выполнения программы, установки точек останова (breakpoints) и инспекции переменных в памяти.
- Менеджер расширений — подсистема интеграции плагинов (например, для поддержки UML и SysMLv2).

Основные приемы работы с редактором кода

Эффективное написание кода и снижение числа ошибок обеспечиваются следующими встроенными технологиями редактора:

- Автодополнение (IntelliSense) — контекстные подсказки имен функций, классов и типов данных на лету при вводе первых букв.
- Подсветка синтаксиса — цветовое выделение ключевых слов языка, типов данных, строк и комментариев для улучшения читаемости кода.
- Фоновый статический анализ (Линтинг) — мгновенное обнаружение и подчеркивание синтаксических ошибок (например, несовместимости типов в WinAPI) еще до запуска компиляции.
- Навигация и рефакторинг — быстрое перемещение к объявлению функций (клавиша F12) и безопасное массовое переименование переменных по всему проекту.

Код программы

Main.cpp

```
#include <QApplication>
#include "MainWindow.h"

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    MainWindow window;
    window.show();

    return app.exec();
}
```

InputDialog.cpp

```
#include "InputDialog.h"

#include <QLineEdit>
#include <QComboBox>
#include <QDialogButtonBox>
#include <QFormLayout>
#include <QVBoxLayout>

InputDialog::InputDialog(QWidget *parent) : QDialog(parent)
{
    setWindowTitle(tr("Ввод данных"));

    m_valueEdit = new QLineEdit(this);
    m_valueEdit->setPlaceholderText(tr("Например: 75.5"));

    m_sourceCombo = new QComboBox(this);
    m_targetCombo = new QComboBox(this);
    fillCombo(m_sourceCombo);
    fillCombo(m_targetCombo);
    if (m_targetCombo->count() > 1)
        m_targetCombo->setCurrentIndex(1); // по умолчанию цель
отличается от источника

    auto *form = new QFormLayout;
    form->addRow(tr("Вес:"), m_valueEdit);
    form->addRow(tr("Единица (откуда):"), m_sourceCombo);
    form->addRow(tr("Единица (куда):"), m_targetCombo);

    auto *buttons = new QDialogButtonBox(QDialogButtonBox::Ok |
QDialogButtonBox::Cancel, this);
    connect(buttons, &QDialogButtonBox::accepted, this,
&QDialog::accept);
    connect(buttons, &QDialogButtonBox::rejected, this,
&QDialog::reject);

    auto *layout = new QVBoxLayout(this);
    layout->addLayout(form);
    layout->addWidget(buttons);
}

void InputDialog::fillCombo(QComboBox *combo)
{
    for (WeightUnit u : WeightModel::allUnits())
        combo->addItem(WeightModel::unitName(u), static_cast<int>(u));
}
```

```

        void InputDialog::setLastData(const QString &value, WeightUnit
sourceUnit, WeightUnit targetUnit)
        {
            m_valueEdit->setText(value);

            const int srcIndex = m_sourceCombo-
>findData(static_cast<int>(sourceUnit));
            if (srcIndex >= 0)
                m_sourceCombo->setCurrentIndex(srcIndex);

            const int tgtIndex = m_targetCombo-
>findData(static_cast<int>(targetUnit));
            if (tgtIndex >= 0)
                m_targetCombo->setCurrentIndex(tgtIndex);
        }

        QString InputDialog::enteredValue() const
        {
            return m_valueEdit->text();
        }

        WeightUnit InputDialog::sourceUnit() const
        {
            return static_cast<WeightUnit>(m_sourceCombo-
>currentData().toInt());
        }

        WeightUnit InputDialog::targetUnit() const
        {
            return static_cast<WeightUnit>(m_targetCombo-
>currentData().toInt());
        }

```

InputDialog.h

```

#pragma once
#include <QDialog>
#include "WeightModel.h"

class QLineEdit;
class QComboBox;

class InputDialog : public QDialog
{
    Q_OBJECT
public:
    explicit InputDialog(QWidget *parent = nullptr);

    // Восстановление последних введенных данных при повторном открытии
    void setLastData(const QString &value, WeightUnit sourceUnit,
WeightUnit targetUnit);

    QString enteredValue() const;
    WeightUnit sourceUnit() const;
    WeightUnit targetUnit() const;

private:
    QLineEdit *m_valueEdit;
    QComboBox *m_sourceCombo;
    QComboBox *m_targetCombo;

    void fillCombo(QComboBox *combo);
};

```

MainWindow.cpp

```
#include "MainWindow.h"
#include "WeightController.h"
#include "InputDialog.h"

#include <QPushButton>
#include <QLabel>
#include <QVBoxLayout>
#include <QWidget>
#include <QMessageBox>

MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent)
{
    setWindowTitle(tr("Конвертер веса"));

    m_model = new WeightModel(this);
    m_controller = new WeightController(m_model, this);

    connect(m_model, &WeightModel::dataChanged, this,
&MainWindow::onModelDataChanged);
    connect(m_model, &WeightModel::errorOccurred, this,
&MainWindow::onModelError);

    m_enterButton = new QPushButton(tr("Ввести данные"), this);
    connect(m_enterButton, &QPushButton::clicked, this,
&MainWindow::onEnterDataClicked);

    m_inputLabel = new QLabel(tr("Данные еще не введены."), this);
    m_resultLabel = new QLabel(this);
    m_allUnitsLabel = new QLabel(this);
    m_allUnitsLabel->setWordWrap(true);

    auto *central = new QWidget(this);
    auto *layout = new QVBoxLayout(central);
    layout->addWidget(m_enterButton);
    layout->addWidget(m_inputLabel);
    layout->addWidget(m_resultLabel);
    layout->addWidget(m_allUnitsLabel);
    layout->addStretch();
    setCentralWidget(central);

    resize(420, 280);
}

void MainWindow::onEnterDataClicked()
{
    InputDialog dialog(this);

    // Восстановление последних введенных данных при повторном открытии
окна
    if (m_hasLastData)
        dialog.setLastData(m_lastRawValue, m_lastSourceUnit,
m_lastTargetUnit);

    if (dialog.exec() == QDialog::Accepted) {
        m_lastRawValue = dialog.enteredValue();
        m_lastSourceUnit = dialog.sourceUnit();
        m_lastTargetUnit = dialog.targetUnit();
        m_hasLastData = true;

        // Контроллер передает "сырые" данные в модель
        m_controller->handleUserInput(m_lastRawValue, m_lastSourceUnit,
m_lastTargetUnit);
    }
}
```

```

    }
}

void MainWindow::onModelDataChanged()
{
    m_inputLabel->setText(tr("Введено: %1 %2")
        .arg(m_model->lastValue())
        .arg(WeightModel::unitName(m_model->lastSourceUnit())));

    m_resultLabel->setText(tr("Результат: %1 %2")
        .arg(m_model->convertedValue(), 0, 'f', 4)
        .arg(WeightModel::unitName(m_model->lastTargetUnit())));

    // Дополнительно показываем эквивалент во всех поддерживаемых
    единицах -
    // наглядно демонстрирует все "упомянутые величины" из задания.
    const double kg = WeightModel::toKilograms(m_model->lastValue(),
m_model->lastSourceUnit());
    QString allText = tr("Эквивалент во всех единицах:\n");
    for (WeightUnit u : WeightModel::allUnits()) {
        allText += QString(" %1: %2\n")
            .arg(WeightModel::unitName(u))
            .arg(WeightModel::fromKilograms(kg, u), 0, 'f', 4);
    }
    m_allUnitsLabel->setText(allText);
}

void MainWindow::onModelError(const QString &message)
{
    QMessageBox::warning(this, tr("Ошибка"), message);
}

```

MainWindow.h

```

#pragma once
#include <QMainWindow>
#include "WeightModel.h"

class QLabel;
class QPushButton;
class WeightController;

class MainWindow : public QMainWindow
{
    Q_OBJECT
public:
    explicit MainWindow(QWidget *parent = nullptr);

private slots:
    void onEnterDataClicked();
    void onModelDataChanged();
    void onModelError(const QString &message);

private:
    WeightModel *m_model;
    WeightController *m_controller;

    QPushButton *m_enterButton;
    QLabel *m_inputLabel;
    QLabel *m_resultLabel;
    QLabel *m_allUnitsLabel;

    // Запоминание последних введенных данных для восстановления в
    диалоге

```

```

    QString m_lastRawValue;
    WeightUnit m_lastSourceUnit = WeightUnit::Kilogram;
    WeightUnit m_lastTargetUnit = WeightUnit::Pound;
    bool m_hasLastData = false;
};

```

WeightController.cpp

```

#include "WeightController.h"

WeightController::WeightController(WeightModel *model, QObject *parent)
    : QObject(parent), m_model(model)
{
}

void WeightController::handleUserInput(const QString &rawValue,
WeightUnit sourceUnit, WeightUnit targetUnit)
{
    QString normalized = rawValue.trimmed();
    normalized.replace(',', '.', '.');

    bool ok = false;
    const double value = normalized.toDouble(&ok);

    if (!ok || normalized.isEmpty()) {
        // Ошибка формата — контроллер не лезет во внутреннее состояние
        // модели
        // напрямую, а использует тот же канал оповещения View, что и
        // модель.
        emit m_model->errorOccurred(QObject::tr("Введите корректное
число (например: 75.5)."));
        return;
    }

    // Дальнейшую проверку (например, что число положительное)
    // и оповещение View берет на себя сама модель.
    m_model->setInput(value, sourceUnit, targetUnit);
}

```

WeightController.h

```

#pragma once
#include <QObject>
#include "WeightModel.h"

class WeightController : public QObject
{
    Q_OBJECT
public:
    explicit WeightController(WeightModel *model, QObject *parent =
nullptr);

    void handleUserInput(const QString &rawValue, WeightUnit sourceUnit,
WeightUnit targetUnit);

private:
    WeightModel *m_model;
};

```

WeightModel.cpp

```

#include "WeightModel.h"

namespace {

```

```

// Коэффициенты пересчета одной единицы веса в килограммы.
// Забавные единицы взяты как усредненные справочные значения.
double factorToKg(WeightUnit unit)
{
    switch (unit) {
        case WeightUnit::Kilogram:      return 1.0;
        case WeightUnit::Pound:         return 0.453592;    // 1 фунт
        case WeightUnit::PolarBear:     return 450.0;       // средний вес
белого медведя
        case WeightUnit::AfricanElephant: return 6000.0;    // средний вес
африканского слона
        case WeightUnit::Zhiguli:       return 1030.0;     // средний вес
автомобиля ВАЗ "Жигули"
        case WeightUnit::BeerCan:       return 0.5;        // банка пива
0.5 л
        case WeightUnit::Chicken:       return 2.0;        // средний вес
курицы (бройлер)
        case WeightUnit::PaperSheet:    return 0.005;      // лист бумаги
А4, 80 г/м²
    }
    return 1.0;
}

} // namespace

WeightModel::WeightModel(QObject *parent) : QObject(parent) {}

double WeightModel::toKilograms(double value, WeightUnit unit)
{
    return value * factorToKg(unit);
}

double WeightModel::fromKilograms(double kg, WeightUnit unit)
{
    return kg / factorToKg(unit);
}

QString WeightModel::unitName(WeightUnit unit)
{
    switch (unit) {
        case WeightUnit::Kilogram:      return QObject::tr("Килограмм");
        case WeightUnit::Pound:         return QObject::tr("Фунт");
        case WeightUnit::PolarBear:     return QObject::tr("Белый
медведь");
        case WeightUnit::AfricanElephant: return QObject::tr("Африканский
слон");
        case WeightUnit::Zhiguli:       return QObject::tr("Жигули");
        case WeightUnit::BeerCan:       return QObject::tr("Банка пива");
        case WeightUnit::Chicken:       return QObject::tr("Цыпленок");
        case WeightUnit::PaperSheet:    return QObject::tr("Лист бумаги");
    }
    return QString();
}

QList<WeightUnit> WeightModel::allUnits()
{
    return {
        WeightUnit::Kilogram,
        WeightUnit::Pound,
        WeightUnit::PolarBear,
        WeightUnit::AfricanElephant,
        WeightUnit::Zhiguli,
        WeightUnit::BeerCan,
        WeightUnit::Chicken,
    };
}

```

```

        WeightUnit::PaperSheet
    };
}

void WeightModel::setInput(double value, WeightUnit sourceUnit,
WeightUnit targetUnit)
{
    if (value <= 0.0) {
        // Модель сама обнаруживает некорректные данные и сама сообщает
об этом
        emit errorOccurred(QObject::tr("Вес должен быть положительным
числом."));
        return;
    }

    m_lastValue = value;
    m_lastSourceUnit = sourceUnit;
    m_lastTargetUnit = targetUnit;

    const double kg = toKilograms(value, sourceUnit);
    m_convertedValue = fromKilograms(kg, targetUnit);
    m_hasData = true;

    emit dataChanged();
}

```

WeightModel.h

```

#pragma once
#include <QObject>
#include <QString>
#include <QList>

// Поддерживаемые единицы измерения веса
enum class WeightUnit {
    Kilogram,
    Pound,
    PolarBear,
    AfricanElephant,
    Zhiguli,
    BeerCan,
    Chicken,
    PaperSheet
};

//
class WeightModel : public QObject
{
    Q_OBJECT
public:
    explicit WeightModel(QObject *parent = nullptr);

    // Установить новые входные данные. Модель сама валидирует
// и сама уведомляет подписчиков о результате.
    void setInput(double value, WeightUnit sourceUnit, WeightUnit
targetUnit);

    double lastValue() const { return m_lastValue; }
    WeightUnit lastSourceUnit() const { return m_lastSourceUnit; }
    WeightUnit lastTargetUnit() const { return m_lastTargetUnit; }
    double convertedValue() const { return m_convertedValue; }
    bool hasData() const { return m_hasData; }

    // Вспомогательные статические функции пересчета

```

```

static double toKilograms(double value, WeightUnit unit);
static double fromKilograms(double kg, WeightUnit unit);
static QString unitName(WeightUnit unit);
static QList<WeightUnit> allUnits();

signals:
    // Активное оповещение подписчиков (View) об изменении состояния
    void dataChanged();
    void errorOccurred(const QString &message);

private:
    double m_lastValue = 0.0;
    WeightUnit m_lastSourceUnit = WeightUnit::Kilogram;
    WeightUnit m_lastTargetUnit = WeightUnit::Kilogram;
    double m_convertedValue = 0.0;
    bool m_hasData = false;
};

```

Вывод программы

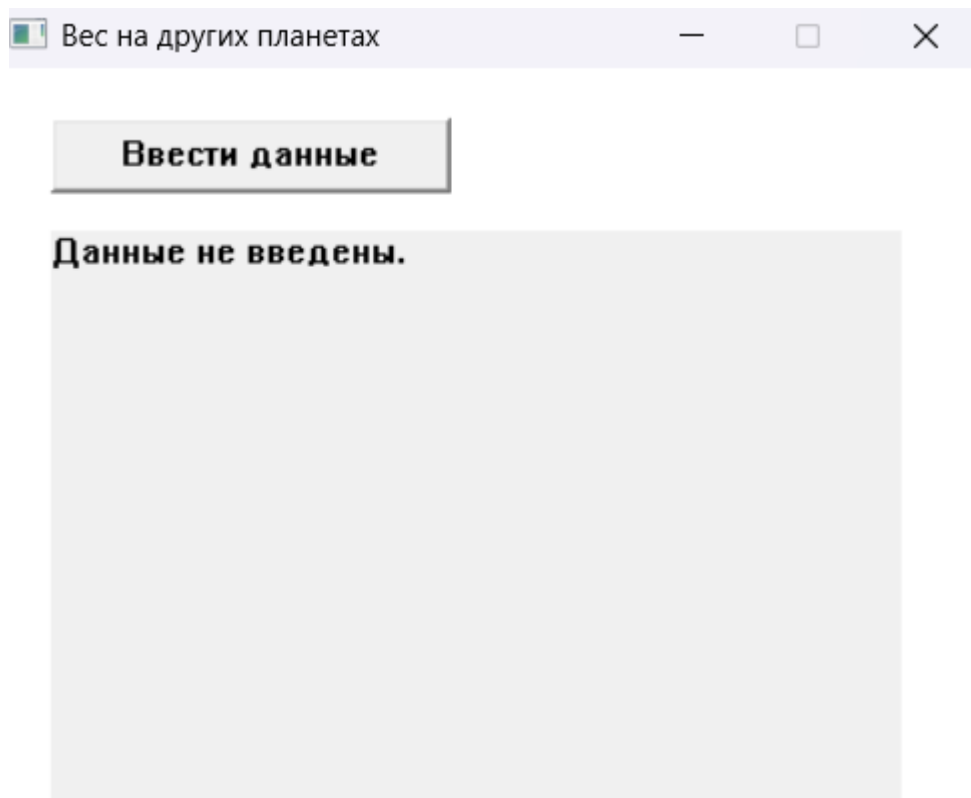


Рисунок 1 – Главный экран

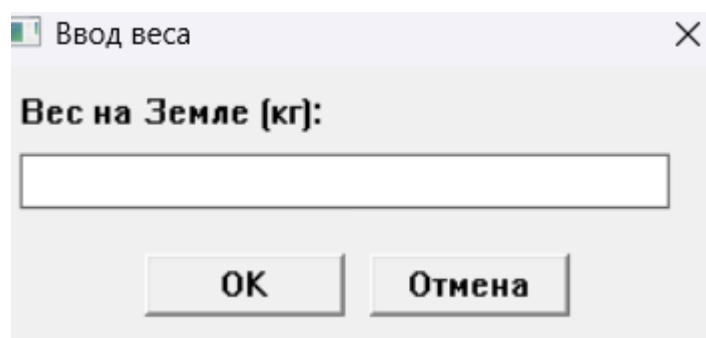
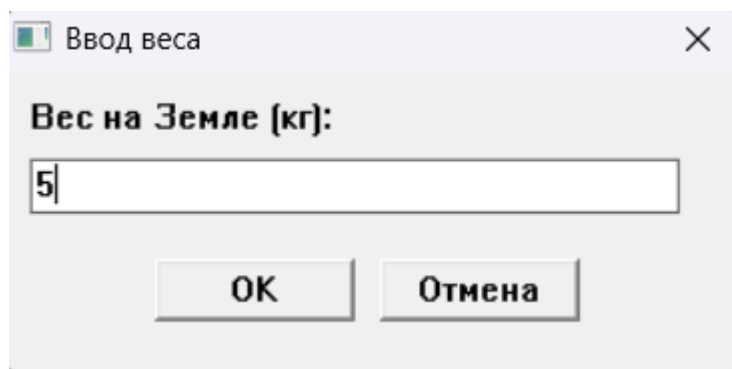


Рисунок 2 – Меню для ввода данных



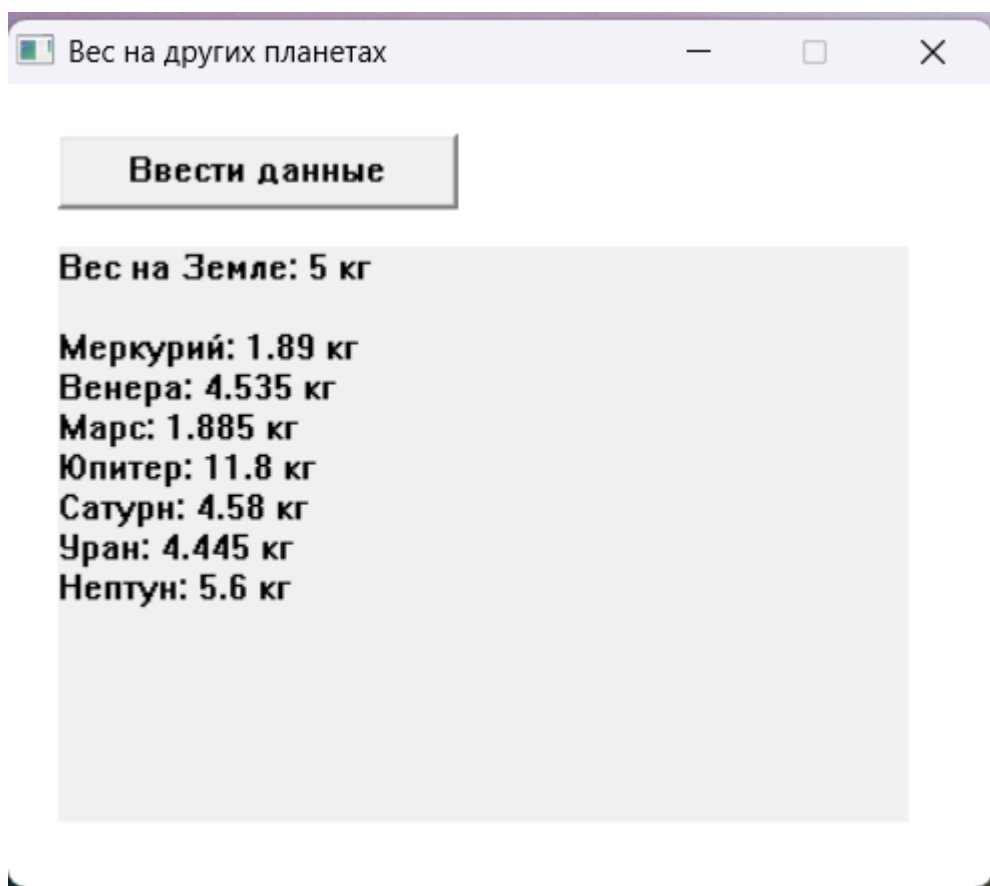
Ввод веса

Вес на Земле (кг):

5

ОК Отмена

Рисунок 3 – Меню после ввода данных



Вес на других планетах

Ввести данные

Вес на Земле: 5 кг

Меркурий: 1.89 кг

Венера: 4.535 кг

Марс: 1.885 кг

Юпитер: 11.8 кг

Сатурн: 4.58 кг

Уран: 4.445 кг

Нептун: 5.6 кг

Рисунок 4 – Главный экран после заполнения данными

Заключение

В результате выполнения лабораторной работы разработано работоспособное приложение с графическим интерфейсом, полностью реализующее функционал заданного варианта — калькулятор веса на планетах Солнечной системы. Реализован пересчёт введённого пользователем веса на Земле в значения веса для других планет, вывод итогового результата на главную форму приложения, а также механизм запоминания и восстановления последних введённых данных при повторном открытии диалогового окна

ввода. Предусмотрена обработка некорректных данных с выводом соответствующего сообщения об ошибке.

Приложение построено на архитектурном шаблоне MVC с активной моделью: модель самостоятельно оповещает представление об изменении внутренних данных посредством механизма подписки, реализованного по паттерну «Наблюдатель» (Observer). Такой подход обеспечивает автоматическое обновление отображаемой информации при изменении состояния модели без необходимости явного опроса её состояния со стороны представления. Программная реализация выполнена на объектно-ориентированном языке C++ с использованием WinAPI, что позволило полностью отделить вычислительную логику модели от компонентов графического интерфейса и реализовать приложение без привлечения внешних GUI-библиотек.